

Prompt:

8/17/2026, 10:01:13 PM

lanjut Tahap P

Response:

8/17/2026, 10:01:14 PM

Tahap P — Execution & Market Microstructure Optimization

Tahap P adalah tahap yang menurut saya **sangat penting** jika tujuan akhirnya memang bukan sekadar membuat *portfolio optimizer*, tetapi membangun sistem **quantitative short-horizon trading** yang realistis.

Sampai Tahap O, kita sudah bisa mengatakan:

"Saya ingin LONG BTC dengan risk budget \$1, target tertentu, dan stop-loss tertentu."

Tetapi pasar tidak bekerja seperti itu.

Kita tidak bisa begitu saja menganggap:

$$P_{entry} = P_t.$$

Dalam kenyataan, kita harus menjawab:

- Apakah menggunakan **market order** atau **limit order**?
- Kalau limit order, di harga berapa?
- Berapa probabilitas order tersebut terisi?
- Berapa lama kita bersedia menunggu?
- Jika belum terisi, apakah order dipindahkan?
- Berapa *slippage*?
- Berapa *spread*?
- Berapa besar order yang sedang mengantre di depan kita?
- Apakah order book sedang menunjukkan tekanan beli/jual?
- Jika order kita tidak filled, apakah peluang pasar bergerak tanpa kita lebih buruk daripada menunggu?
- Bagaimana execution memengaruhi \$1 risk budget kita?

Dengan demikian Tahap P memperkenalkan:

Market Microstructure

ke dalam sistem.

P.1. Perubahan paradigma

Sebelumnya kita memiliki:

Decision Price

seolah-olah harga entry adalah:

$$P_t.$$

Sekarang kita harus membedakan:

$$P_t^{mid}$$

$$P_t^{bid}$$

$$P_t^{ask}$$

$$P_t^{exec}.$$

Dengan:

$$P_t^{mid} = \frac{P_t^{bid} + P_t^{ask}}{2}.$$

Dan:

$$P_t^{ask} - P_t^{bid} = S_t$$

adalah bid-ask spread.

P.2. Entry signal $\neq$ execution

Ini merupakan prinsip utama Tahap P:

Signal Generation $\boxtimes$ Order Execution.

Model sebelumnya mungkin mengatakan:

LONG.

Tetapi execution optimizer harus menentukan:

HOW to become LONG.

Misalnya:

BUY MARKET,
BUY LIMIT @P_{bid},
BUY LIMIT @P_{bid} - 1tick,
BUY LIMIT @P_{bid} - 2ticks,
WAIT.

P.3. State market microstructure

Sekarang state kita diperluas.

Sebelumnya:

$$s_t = \{P_t, \mu_t, \Sigma_t, \Theta_t, \dots\}.$$

Sekarang:

$$s_t^{micro} = \{P_t^{bid}, P_t^{ask}, V_t^{bid}, V_t^{ask}, S_t, I_t, D_t, Q_t, \lambda_t, \sigma_t\}.$$

Beberapa variabel penting:

- V_t^{bid} : volume bid,
 - V_t^{ask} : volume ask,
 - S_t : spread,
 - I_t : order-book imbalance,
 - D_t : depth,
 - Q_t : queue position,
 - λ_t : order arrival intensity,
 - σ_t : short-term volatility.
-

P.4. Order-book imbalance

Salah satu feature paling dasar adalah:

$$I_t = \frac{V_t^{bid} - V_t^{ask}}{V_t^{bid} + V_t^{ask}}$$

sehingga:

$$-1 \leq I_t \leq 1.$$

Jika:

$$I_t \rightarrow 1,$$

bid jauh lebih besar daripada ask.

Jika:

$$I_t \rightarrow -1,$$

ask jauh lebih besar daripada bid.

P.5. Tetapi imbalance sederhana belum cukup

Misalnya:

$$V_1^{bid} = 100, \quad V_1^{ask} = 20.$$

Kita mendapatkan:

$$I = 0.667.$$

Tetapi mungkin volume tersebut berada sangat jauh dari mid-price.

Karena itu kita dapat menggunakan **depth-weighted imbalance**.

Misalkan terdapat K level:

$$k = 1, \dots, K.$$

Maka:

$$I_t^{(K)} = \frac{\sum_{k=1}^K w_k V_{bid,k} - \sum_{k=1}^K w_k V_{ask,k}}{\sum_{k=1}^K w_k V_{bid,k} + \sum_{k=1}^K w_k V_{ask,k}}$$

dengan:

$$w_k = e^{-\alpha(k-1)}.$$

Level yang lebih dekat dengan mid mendapat bobot lebih besar.

P.6. Microprice

Mid-price:

$$M_t = \frac{A_t + B_t}{2}.$$

Tetapi mid-price tidak memperhitungkan imbalance.

Kita dapat menggunakan:

$$P_t^{micro} = \frac{A_t V_t^{bid} + B_t V_t^{ask}}{V_t^{bid} + V_t^{ask}}$$

dengan:

- A_t : best ask,
- B_t : best bid.

Jika:

$$V^{bid} \gg V^{ask},$$

maka:

$$P^{micro}$$

cenderung mendekati ask.

Ini memberi estimasi yang lebih informatif mengenai tekanan harga jangka sangat pendek.

P.7. Short-horizon expected price

Sekarang kita dapat membuat:

$$\hat{P}_{t+\Delta} = E[P_{t+\Delta} | F_t].$$

Atau expected return:

$$\hat{r}_{t,\Delta} = E \left[\ln \frac{P_{t+\Delta}}{P_t} \middle| F_t \right].$$

Dan model dapat menggunakan:

$$\hat{r} = f(I_t, P_t^{micro} - P_t^{mid}, \sigma_t, \text{factor state, momentum, flow}).$$

P.8. Sekarang masalah utama: fill probability

Misalkan kita ingin membeli.

Kita mempunyai dua pilihan:

Market order

Probability fill:

$$P(\text{Fill}) \approx 1.$$

Tetapi:

$$Cost_{\text{execution}}$$

lebih besar.

Limit order

Cost lebih rendah.

Tetapi:

$$P(\text{Fill}) < 1.$$

Maka execution sebenarnya adalah trade-off:

$$\boxed{\text{Execution Cost} \leftrightarrow \text{Fill Probability.}}$$

P.9. Expected execution utility

Misalkan action:

$$a$$

adalah jenis order.

Kita definisikan:

$$p_F(a) = P(\text{fill} \mid a).$$

Jika filled:

$$U_F(a).$$

Jika tidak filled:

$$U_N(a).$$

Maka:

$$\boxed{E[U(a)] = p_F(a)U_F(a) + (1 - p_F(a))U_N(a).}$$

Ini merupakan fondasi execution optimizer kita.

P.10. Mengapa "tidak filled" bukan utility nol?

Ini sangat penting.

Misalkan model sangat yakin:

$$P_{t+\Delta} > P_t.$$

Kita menaruh limit buy:

$$P_L < P_t.$$

Jika tidak filled dan harga naik, kita kehilangan kesempatan.

Opportunity cost tersebut adalah:

$$U_N < 0.$$

Jadi limit order murah belum tentu optimal.

P.11. Market order versus limit order

Misalkan:

Market

$$p_F = 0.999.$$

Expected execution price:

$$P_M = P_t + \text{slippage}.$$

Limit

$$p_F = 0.65.$$

Execution price:

$$P_L < P_t.$$

Tetapi jika tidak filled:

$$P_{\text{future}} > P_t.$$

Maka:

$$E[U_{\text{limit}}]$$

bisa saja lebih rendah daripada:

$$E[U_{\text{market}}].$$

P.12. Implementation shortfall

Kita memerlukan ukuran:

$$IS$$

atau implementation shortfall.

Untuk long secara sederhana:

$$IS = P_{exec} - P_{decision}$$

dalam satuan harga.

Jika:

$$P_{exec} > P_{decision},$$

maka execution lebih buruk.

Dalam return:

$$IS_r = \frac{P_{exec} - P_{decision}}{P_{decision}}.$$

P.13. Total execution cost

Kita dapat memecah:

$$C_{exec} = C_{spread} + C_{slippage} + C_{fee} + C_{impact} + C_{opportunity}.$$

Perhatikan:

$$C_{opportunity}$$

berasal dari order yang tidak terisi.

Jadi limit order tidak benar-benar "gratis".

P.14. Spread cost

Jika kita membeli menggunakan market order:

$$P_{ask}$$

sedangkan reference price:

$$P_{mid}.$$

Maka:

$$C_{spread} = P_{ask} - P_{mid} = \frac{S}{2}.$$

Untuk sell:

$$P_{mid} - P_{bid} = \frac{S}{2}.$$

P.15. Slippage

Misalkan kita ingin membeli:

$$q = 100.$$

Tetapi best ask hanya memiliki:

$$40.$$

Order kita harus mengambil:

$$40$$

pada level pertama,

sisanya:

$$60$$

pada level berikutnya.

Execution price menjadi weighted average:

$$P_{VWAP} = \frac{\sum_k P_k q_k}{\sum_k q_k}.$$

Slippage:

$$Slippage = P_{VWAP} - P_{best}.$$

P.16. Market impact

Jika:

$$q$$

cukup besar relatif terhadap liquidity, order kita sendiri mengubah harga.

Secara sederhana:

$$Impact(q) \propto \left(\frac{q}{D}\right)^\alpha$$

dengan:

- D : available liquidity,
- $\alpha > 0$.

Model yang lebih realistis dapat menggunakan empirical impact function.

P.17. Risk budget sekarang harus memasukkan execution

Sebelumnya:

$$R = q|P_{entry} - P_{SL}|.$$

Sekarang entry tidak diketahui secara pasti.

Kita harus menggunakan:

$$P_{exec}.$$

Maka:

$$R = q|P_{exec} - P_{SL}| + C_{exit} + C_{entry} + C_{slippage}.$$

Dan constraint kita tetap:

$$R \leq \$1.$$

Ini sangat penting.

P.18. Execution price uncertainty

Kita bahkan dapat menganggap:

$$P_{exec}$$

sebagai random variable.

$$P_{exec} \sim p(P \mid a, s_t).$$

Maka risk menjadi:

$$R(a) = E[q|P_{exec} - P_{SL}|] + C(a).$$

Tetapi untuk sistem risk-sensitive, expected value saja belum cukup.

Kita dapat menggunakan:

$$VaR$$

atau:

$$CVaR.$$

P.19. CVaR execution risk

Misalkan:

$$L_{exec}$$

adalah execution loss.

Kita dapat membatasi:

$$CVaR_{\alpha}(L_{exec}) \leq L_{max}.$$

Dengan demikian order yang memiliki tail slippage besar dapat ditolak meskipun expected slippage kecil.

P.20. Queue position

Sekarang masuk bagian yang sangat menarik.

Misalkan kita menaruh:

BUY LIMIT

di:

$$P = B_t.$$

Di depan order kita terdapat:

$$Q_t$$

unit.

Agar order kita filled, market harus mengonsumsi:

$$Q_t + q$$

volume pada harga tersebut, kecuali terdapat cancelation.

Maka:

$$P(\text{Fill}) = P(V_{consumed} \geq Q_t + q).$$

P.21. Queue depletion model

Misalkan order consumption mengikuti counting process:

$$N_t.$$

Kita dapat memodelkannya sebagai Poisson:

$$N_t \sim \text{Pois}(\lambda t).$$

Probability order terisi:

$$P(\text{Fill} \leq t) = P(N_t \geq Q_t + q).$$

Secara eksplisit:

$$P(\text{Fill} \leq t) = 1 - \sum_{k=0}^{Q_t+q-1} e^{-\lambda t} \frac{(\lambda t)^k}{k!}.$$

Ini memberikan hubungan:

$$\text{Queue} \rightarrow \text{Fill Probability}.$$

Model Poisson ini adalah **baseline**, bukan asumsi final; order flow nyata biasanya lebih kompleks dan dapat menunjukkan clustering serta self-excitation.

P.22. Cancellation rate

Queue tidak hanya berkurang karena transaksi.

Order di depan kita dapat dibatalkan.

Misalkan:

$$\mu$$

adalah cancellation intensity.

Maka effective queue depletion:

$$\lambda_{\text{effective}} = \lambda_{\text{trade}} + \lambda_{\text{cancel}}.$$

Sehingga:

$$P(\text{Fill})$$

meningkat jika cancellation rate meningkat.

P.23. Expected time to fill

Dengan intensity:

$$\lambda_{eff},$$

secara sederhana:

$$E[T_{fill}] \approx \frac{Q_t + q}{\lambda_{eff}}.$$

Tetapi dalam sistem nyata kita sebaiknya menggunakan distribusi:

$$T_{fill} \sim F_T(t \mid s_t, a).$$

P.24. Time-to-fill menjadi decision variable

Sekarang kita dapat menentukan:

$$\tau_{max}.$$

Misalnya:

$$\tau_{max} = 15 \text{ menit.}$$

Jika:

$$T_{fill} > \tau_{max},$$

order dianggap tidak lagi menarik.

Tetapi τ_{max} sendiri seharusnya dioptimalkan.

P.25. Dynamic order placement

Kita sekarang punya kandidat:

$$a \in A_{exec}.$$

Misalnya:

$$A_{exec} = \{MARKET, LIMIT_0, LIMIT_1, LIMIT_2, WAIT\}.$$

Dengan:

$$LIMIT_k = P_{bid} - k\Delta P.$$

P.26. Objective execution

Untuk setiap action:

$$a,$$

kita menghitung:

$$U_{exec}(a) = E[\Pi \mid a] - C_{exec}(a) - \lambda_r R(a) - \lambda_o OC(a).$$

Kemudian:

$$a^* = \arg \max_a U_{exec}(a).$$

P.27. Tetapi execution dan alpha saling berinteraksi

Misalkan:

$$\hat{r}_{5min} = +0.10\%.$$

Spread:

$$0.02\%.$$

Limit order memberi expected price improvement:

$$0.015\%.$$

Tetapi probability fill hanya:

30%.

Jika harga diperkirakan naik cepat, waiting cost bisa lebih besar daripada price improvement.

Jadi execution optimizer harus mengetahui:

alpha decay.

P.28. Alpha decay

Misalkan expected return:

$$f(\tau)$$

berkurang terhadap waktu:

$$f(\tau) = f_0 e^{-\kappa \tau}.$$

Parameter:

$$\kappa$$

adalah alpha decay rate.

Jika:

$$\kappa$$

besar, kita harus cepat.

Jika:

$$\kappa$$

kecil, kita dapat menunggu limit fill.

P.29. Ini menghasilkan trade-off yang sangat elegan

Limit order memberikan:

Price Improvement.

Tetapi menunggu memberikan:

Alpha Decay.

Maka:

$$\text{Execution Optimization} = \text{Price Improvement} - \text{Waiting Cost}$$

dengan waiting cost bergantung pada:

α decay.

P.30. Opportunity cost akibat non-fill

Misalkan jika tidak filled dalam waktu τ , kita kehilangan expected move:

$$E[\Delta P_\tau].$$

Maka:

$$OC(\tau) = P(\text{Fill}^c, \tau) \cdot E[\Delta P_\tau].$$

Untuk long dan expected move positif, waiting bisa sangat mahal.

P.31. Limit order placement sebagai optimasi

Kita dapat memilih:

$$\delta$$

sebagai jarak limit dari mid:

$$P_L = P_{mid} - \delta.$$

Kemudian:

$$\delta^* = \arg \max_{\delta} U_{exec}(\delta).$$

Dengan:

$$U_{exec}(\delta) = p_F(\delta)U_F(\delta) + [1 - p_F(\delta)]U_N(\delta).$$

Ini merupakan salah satu core problem Tahap P.

P.32. Sekarang kita bisa membuat execution QUBO

Diskretkan:

$$\delta \in \{0, 1, 2, \dots, K\}.$$

Definisikan binary:

$$x_k \in \{0, 1\}.$$

Dengan:

$$\sum_{k=0}^K x_k = 1.$$

Kemudian:

$$\delta = \sum_{k=0}^K \delta_k x_k.$$

P.33. Objective

Jika:

$$U_k = U_{exec}(\delta_k),$$

maka:

$$E(x) = - \sum_k U_k x_k + P \left(\sum_k x_k - 1 \right)^2 .$$

Ini sudah menjadi QUBO.

P.34. Jika kita juga mengoptimalkan order size

Sekarang:

$$q$$

juga menjadi decision variable.

Misalnya:

$$q \in \{0.25q^*, 0.5q^*, 0.75q^*, q^*\}.$$

Kita dapat menggunakan:

$$x_{k,m}$$

untuk kombinasi:

$$(\delta_k, q_m).$$

Kemudian:

$$(\delta^*, q^*) = \arg \max_{\delta, q} U_{exec}(\delta, q).$$

P.35. Tetapi quantity dibatasi risk

Constraint:

$$q|P_{exec} - SL| \leq \$1.$$

Jika:

$$P_{exec}$$



random, kita dapat menggunakan conservative estimate:

$$q(P_{exec}^{worst} - SL) \leq 1.$$

Atau:

$$CVaR_{\alpha}(R) \leq 1.$$

P.36. Multi-asset execution

Sekarang bayangkan:

$$BTC, ETH, SOL, XRP.$$

Kita ingin:

$$q_i$$

untuk setiap aset.

Total risk:

$$R_p = w^T \Sigma w$$

atau jika menggunakan dollar risk:

$$R_p \leq R_{budget}.$$

Execution optimizer sekarang harus menentukan:

$$\{q_i, \delta_i, a_i\}_{i=1}^N.$$

P.37. Ini kembali menjadi combinatorial optimization

Karena:

$$a_i$$

bisa:

MARKET/LIMIT/WAIT,

dan:

δ_i

diskrit,

kita mendapatkan:

Execution Portfolio Optimization.

Objective:

$$\max \sum_i U_i^{exec} - \lambda Risk_p - \lambda Cost_p.$$

P.38. Cross-asset market impact

Ini menjadi menarik untuk crypto.

Misalnya kita ingin membeli:

SOL.

Tetapi:

BTC

mengalami sell pressure besar.

Maka expected SOL return dapat berubah.

Dengan factor model:

$$r_{SOL} = \beta_{SOL,BTC} r_{BTC} + \epsilon.$$

Jika:

r_{BTC}

berubah, execution decision SOL juga berubah.

P.39. Untuk saham internasional

Kita dapat memperluas factor state:

$$F_t = [DXY, Gold, SPX, Nasdaq, VIX, Treasury, BTC, \dots].$$

Misalnya untuk saham teknologi:

$$r_i = \alpha_i + \beta_{i,DXY} r_{DXY} + \beta_{i,SPX} r_{SPX} + \beta_{i,NQ} r_{NQ} + \epsilon_i.$$

Execution optimizer kemudian mengetahui apakah:

$$\hat{r}_i$$

cukup besar untuk membayar:

$$spread + slippage + impact.$$

P.40. Ini menghasilkan konsep penting: alpha after execution

Alpha mentah:

$$\alpha_{raw}.$$

Execution cost:

$$C_{exec}.$$

Maka:

$$\alpha_{net} = \alpha_{raw} - C_{exec}.$$

Tetapi karena fill probability juga penting:

$$E[\alpha_{net}] = p_F(\alpha_{raw} - C_F) + (1 - p_F)\alpha_{no-fill}.$$

Jadi signal yang terlihat sangat bagus dapat menjadi trade yang buruk setelah execution.

P.41. No-trade region

Ini menghasilkan konsep yang sangat penting.

Jika:

$$\alpha_{raw} < C_{exec},$$

maka:

NO TRADE.

Dengan kata lain:

Model tidak wajib trading hanya karena ada signal.

Kita dapat mendefinisikan:

$$|\rho_t| > C_{exec} + C_{risk} + C_{uncertainty}$$

sebagai syarat minimum untuk melakukan trade.

P.42. Hysteresis execution

Sama seperti Tahap O, jangan terus-menerus memindahkan limit order.

Jika:

$$U_{new} - U_{old} < C_{modify},$$

maka:

KEEP CURRENT ORDER.

Dengan demikian sistem tidak mengalami:

Cancel → Replace → Cancel → Replace.

P.43. Order lifecycle

Sekarang setiap order mempunyai state machine.

$$\boxed{NEW \rightarrow PLACED \rightarrow PARTIAL \rightarrow FILLED}$$

atau:

$$PLACED \rightarrow CANCELLED.$$

Atau:

$$PARTIAL \rightarrow CANCELLED.$$

P.44. Mengapa partial fill penting?

Misalkan target:

$$q^* = 100.$$

Tetapi hanya:

$$40$$

filled.

Maka kita memiliki:

$$q_{filled} = 40.$$

Remaining:

$$q_{remaining} = 60.$$

Namun setelah 40 filled, market state dapat berubah.

Maka:

$$q_{remaining}$$

tidak boleh dianggap otomatis harus dikejar.

Kita harus melakukan re-optimization.

P.45. Partial fill sebagai decision problem

Setelah partial fill:

$$a \in \{KEEP_ORDER, CANCEL, REPRICE, MARKET_REMAINDER, WAIT\}.$$

Dan:

$$a^* = \arg \max_a U(a).$$

P.46. Adverse selection

Ini salah satu masalah paling penting dalam limit order.

Misalkan kita memasang:

BUY LIMIT.

Order kita filled.

Mengapa?

Mungkin karena:

$$P_t$$

justru akan turun.

Artinya:

Fill

sendiri membawa informasi negatif.

Ini disebut **adverse selection**.

P.47. Fill probability tidak selalu positif

Kita sebelumnya menulis:

$$p_F.$$

Tetapi sebenarnya:

$$P(\text{return} > 0 \mid \text{Fill})$$

dapat lebih rendah daripada:

$$P(\text{return} > 0 \mid \text{NoFill}).$$

Jadi kita harus memodelkan:

$$P(\Delta P > 0 \mid \text{Fill}, s_t).$$

Bukan hanya:

$$P(\text{Fill}).$$

P.48. Execution alpha

Kita dapat mendefinisikan:

$$A_{exec} = E[\Delta P \mid \text{Fill}] - E[\Delta P \mid \text{NoFill}].$$

Jika:

$$A_{exec} < 0,$$

fill cenderung terjadi ketika market bergerak melawan kita.

Maka limit order tersebut mungkin memiliki adverse selection yang tinggi.

P.49. Objective execution yang lebih lengkap

Sekarang:

$$U_{exec}(a) = p_F(a) [E[\Pi \mid Fill, a] - C_F(a)] + (1 - p_F(a)) [E[\Pi \mid NoFill, a] - C_N(a)]$$

dan:

$$a^* = \arg \max_a U_{exec}(a).$$

Ini jauh lebih realistis daripada:

"limit order selalu lebih baik karena fee lebih murah."

P.50. Hubungan dengan Tahap O

Tahap O menghasilkan:

P_{entry}^{target} ,

SL ,

TP ,

q .

Tahap P kemudian mengatakan:

Bagaimana kita mendapatkan q tersebut dengan biaya dan risiko execution minimum?

Jadi:

$$O \rightarrow P$$

bukan dua modul independen.

P.51. Execution-aware risk

Misalkan Tahap O mengatakan:

$$q = 100.$$

Tetapi Tahap P memperkirakan:

$$slippage = 0.3\%.$$

Maka actual risk:

$$R_{actual} > R_{planned}.$$

Jika:

$$R_{actual} > \$1,$$

maka execution layer harus mengurangi:

$$q.$$

Jadi quantity final:

$$q_{exec} = \min(q_{optimal}, q_{risk}, q_{liquidity}).$$

P.52. Risk budget hierarchy

Kita sekarang memiliki:

$$R_{total} \leq \$1.$$

Tetapi:

$$R_{total} = R_{market} + R_{execution} + R_{gap} + R_{model}.$$

Sehingga secara konseptual:

$$R_{market} + R_{execution} \leq R_{budget}.$$

Ini lebih realistis daripada mengalokasikan seluruh \$1 kepada price movement saja.

P.53. Dynamic execution budget

Misalnya:

$$R_{budget} = 1.$$

Market risk:

$$R_{market} = 0.75.$$

Maka execution hanya memiliki:

$$R_{execution} \leq 0.25.$$

Jika spread/slippage terlalu besar:

$$R_{execution} > 0.25,$$

trade harus:

REJECTED.

P.54. Ini menghasilkan gate terakhir

Sebelum order dikirim:

Trade Gate

memeriksa:

$$\begin{aligned} EV &> 0, \\ Risk &\leq Budget, \\ Liquidity &\geq Minimum, \\ ExecutionCost &\leq Maximum, \\ FillProbability &\geq Minimum, \\ Regime &valid. \end{aligned}$$

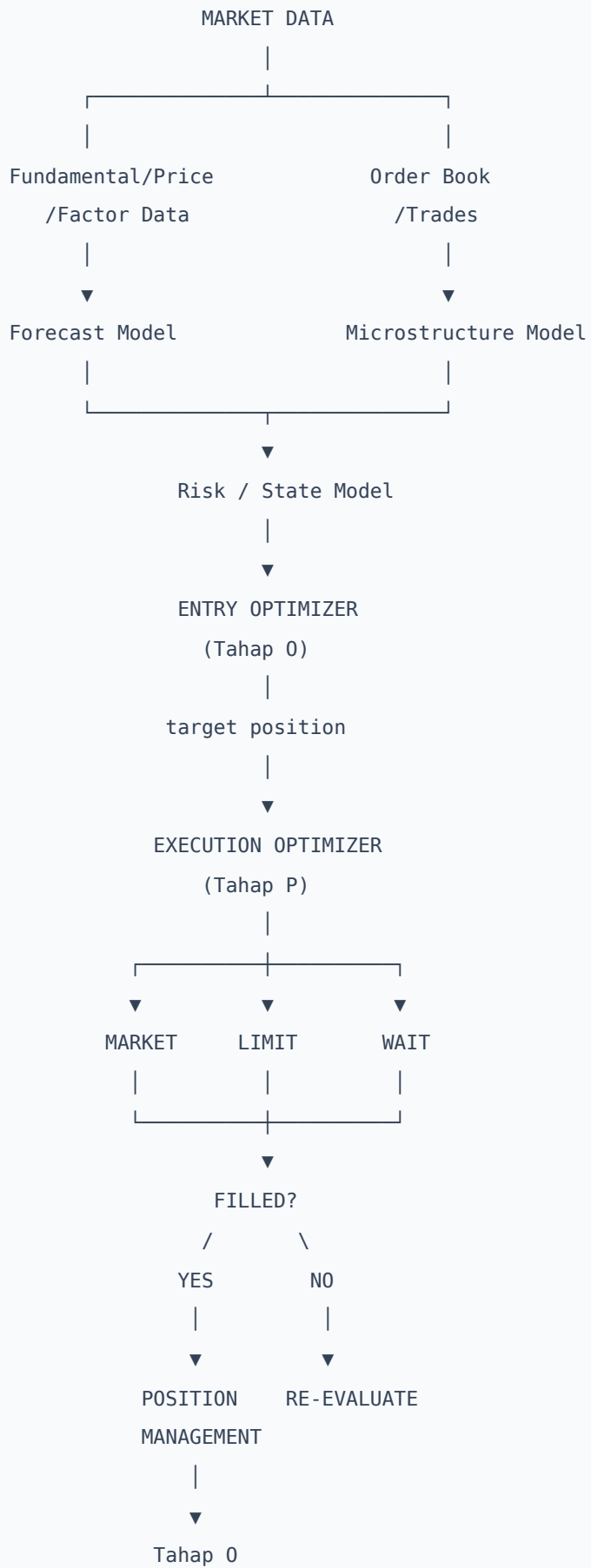
Jika salah satu constraint kritis gagal:

NO TRADE.



P.55. Arsitektur lengkap setelah Tahap P

Sekarang sistem kita menjadi:



P.56. Dan sekarang kita mempunyai tiga lapisan optimasi

Ini mulai menjadi sangat menarik.

Layer 1 — Portfolio / Entry

What should I trade?

QUBO:

Q_{entry} .

Layer 2 — Position Management

What should I do with the trade?

QUBO:

Q_{manage} .

Layer 3 — Execution

How should I execute the decision?

QUBO:

Q_{exec} .

P.57. Tiga QUBO

Secara ringkas:

$Q_{entry} \rightarrow Q_{manage} \rightarrow Q_{exec}$.

Atau:

Selection → Management → Execution.



Dan semuanya menggunakan market state yang diperbarui secara periodik.

P.58. Tetapi saya ingin membuat satu perubahan penting pada desain kita

Jangan membuat:

$$Q_{entry}, Q_{manage}, Q_{exec}$$

sepenuhnya terpisah.

Karena ada coupling.

Contohnya:

entry size

mempengaruhi:

execution impact.

Execution mempengaruhi:

$$P_{entry}.$$

Entry price mempengaruhi:

$$SL.$$

SL mempengaruhi:

risk.

Risk mempengaruhi:

portfolio allocation.

Jadi secara ideal:

Hierarchical Optimization

dengan feedback.

P.59. Mathematical feedback loop

Secara konseptual:

$$x_t^{entry} \rightarrow q_t \rightarrow a_t^{exec} \rightarrow P_t^{exec} \rightarrow R_t \rightarrow x_t^{manage}.$$

Kemudian:

$$x_t^{manage} \rightarrow \text{future capital} \rightarrow x_{t+1}^{entry}.$$

Jadi sistem merupakan:

closed – loop optimization.

P.60. Ini juga membuat proyek kita cocok untuk eksperimen quantum

Karena kita dapat menguji:

Classical baseline

Exact/Brute Force

untuk problem kecil.

Classical heuristic

Simulated Annealing

Tabu Search

MILP

Quantum

QAOA



Kemudian dibandingkan berdasarkan:

*solution quality,
approximation ratio,
runtime,
number of circuit evaluations,
constraint violation,
execution performance.*

P.61. Metrik yang harus kita simpan

Untuk setiap order:

*$P_{decision}$,
 P_{limit} ,
 P_{exec} ,
spread,
slippage,
fee,
queue estimate,
fill probability,
actual fill time,
partial fill,
market move after order.*

Ini akan menjadi **dataset execution** kita.

P.62. Ini membuka kemungkinan self-improving execution model

Setelah banyak trade, kita dapat mengestimasi:

$$P(Fill \mid state, order).$$

Kemudian:

$$E[Slippage \mid state, order].$$

Kemudian:

$$E[\Delta P \mid Fill, state].$$



Sehingga execution model semakin baik.

Jadi:

$$\textit{Trading system} \rightarrow \textit{Execution data} \rightarrow \textit{Better execution model} \rightarrow \textit{Better optimization}.$$

P.63. Tetapi jangan langsung menggunakan reinforcement learning

Untuk tahap awal, saya justru menyarankan:

$$\textit{Supervised/Statistical Models} + \textit{Optimization}$$

daripada:

$RL.$

Karena kita dapat menjelaskan dengan jelas:

$P(Fill),$

$Slippage,$

$Impact,$

$Expected Return,$

dan objective function.

RL dapat kita pertimbangkan jauh kemudian jika diperlukan.

P.64. Pipeline matematis Tahap P

Saya akan merangkumnya sebagai:

$$F_t \rightarrow \textit{Order Book Features}$$

↓

$$P(Fill \mid a, s_t)$$

$$E[Slippage \mid a, s_t]$$

$$E[\Delta P \mid a, s_t]$$

↓

$$U_{exec}(a)$$

↓

$$Q_{exec}$$

↓

$$a^*$$

↓

$$Order\ Placement$$

P.65. Satu persamaan yang merangkum Tahap P

Jika saya harus memilih satu persamaan inti:

$$a_t^* = \arg \max_{a \in A} \{p_F(a) [E[\Pi \mid Fill, a] - C_{exec}(a)] + (1 - p_F(a))E[\Pi \mid NoFill, a] - \lambda R(a)\}.$$

Ini adalah inti **execution optimization** kita.

P.66. Dan constraint paling penting tetap:

$$R(a) \leq \$1.$$

Bahkan setelah memasukkan:

spread,

slippage,

market impact,

fees,

dan:

fill uncertainty.

Jadi target awal kita:

"setiap trade maksimal berisiko \$1"

sekarang menjadi jauh lebih rigor.

Bukan lagi:

$$q(P_{\text{entry}} - SL) = 1,$$

tetapi kira-kira:

$$CVaR_{\alpha} [q|P_{\text{exec}} - SL| + C_{\text{execution}} + C_{\text{exit}}] \leq \$1.$$

P.67. Posisi sistem kita setelah Tahap P

Kita sekarang memiliki:

$$\mathbf{A-O} = \text{Market} \rightarrow \text{Forecast} \rightarrow \text{Risk} \rightarrow \text{Portfolio} \rightarrow \text{Entry} \rightarrow \text{Position Management}$$

dan Tahap P menambahkan:

$$P = \text{Execution} + \text{Order Book} + \text{Fill Probability} + \text{Slippage} + \text{Market Impact}.$$

Sehingga sistem akhirnya mulai menyerupai arsitektur **institutional systematic trading**:

Alpha → Portfolio Construction → Risk → Execution → Position Management

dengan optimasi kombinatorial di beberapa lapisan.

Hal yang paling penting dari Tahap P

Ada satu perubahan filosofis yang menurut saya sangat penting untuk proyek kita:

Jangan menganggap order sebagai instruksi. Anggap order sebagai keputusan ekonomi yang memiliki probabilitas, biaya, dan konsekuensi.

Dengan begitu:

BUY

bukan lagi sebuah aksi sederhana.

Ia menjadi:

(BUY, price, quantity, order type, queue, lifetime)

yang seluruhnya merupakan **decision variables**.

Dan karena sebagian besar variabel tersebut dapat didiskretkan, kita kembali mendapatkan problem:

Combinatorial Optimization → QUBO → Ising → QAOA.

Posisi kita sekarang

Arsitektur besarnya sudah menjadi:



Tahap berikutnya, Tahap Q, sebaiknya kita gunakan untuk membangun bagian yang selama ini menjadi "jembatan" antara semua model tersebut: **Backtesting Engine & Event-Driven Simulation**. Kita perlu membuat dunia virtual yang benar-benar mensimulasikan urutan **data masuk → signal → QUBO → order → partial fill/non-fill → slippage → posisi → dynamic SL/TP → exit**, tanpa *look-ahead bias* dan tanpa *survivorship bias*. Ini krusial sebelum satu baris algoritma quantum pun dianggap memiliki performa trading yang valid.